

Machine Learning Algorithms Laboratory

Experiment 3

Regression Analysis using Linear and Regularized Models

Name	MAHALINGAM S S
Register No.	3122247001034
Degree & Branch	M. Tech Computer Science & Engineering
Semester	VI
Subject Code & Name	ICS1512 – Machine Learning Algorithms Laboratory
Academic Year	2026–2027
Batch	2024–2029

Aim and Objective

To implement Linear Regression, Ridge Regression, Lasso Regression and Elastic Net Regression for predicting loan amounts, compare their performance using different regression metrics and understand the effect of regularization.

- Implement Linear Regression as a baseline regression model.
- Implement Ridge, Lasso and Elastic Net regression.
- Tune regularization hyperparameters using GridSearchCV.
- Evaluate models using MAE, MSE, RMSE and R^2 score.
- Perform five-fold cross-validation.
- Visualize prediction behaviour, residuals, coefficient values and training versus validation error.
- Analyze overfitting, underfitting and the bias–variance trade-off.

Dataset Description

The loan amount dataset was used for this experiment. It contains numerical and categorical features related to loan applications. The target variable is the loan amount sanctioned.

The dataset contains the following relevant attributes:

- Number of dependents
- Education
- Self-employed status
- Annual income
- Loan term
- CIBIL score
- Residential assets value
- Commercial assets value
- Luxury assets value
- Bank asset value
- Loan status
- Loan amount – target variable

Preprocessing Steps

The following preprocessing steps were carried out:

- Loaded the loan dataset.
- Checked dataset shape, column names, data types and summary statistics.
- Checked for missing values.
- Removed the `loan_id` column.
- Encoded categorical variables using `LabelEncoder`.
- Separated the input variables from the target variable `loan_amount`.
- Standardized the input features using `StandardScaler`.
- Split the data into training and testing sets using an 80:20 split with random state 42.

Implementation Details

Four regression models were implemented:

- Linear Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression

Linear Regression was used as the baseline. Ridge, Lasso and Elastic Net were then used to study the effect of regularization.

Linear Regression

Linear Regression models the relationship between input features and a continuous target variable:

$$y = X\beta + \epsilon$$

where β represents the model coefficients and ϵ represents the error term.

Ridge Regression

Ridge uses L2 regularization. It adds a penalty proportional to the squared magnitude of the coefficients:

$$\min_{\beta} (\|y - X\beta\|_2^2 + \alpha\|\beta\|_2^2)$$

It reduces coefficient magnitudes and can reduce variance.

Lasso Regression

Lasso uses L1 regularization:

$$\min_{\beta} (\|y - X\beta\|_2^2 + \alpha\|\beta\|_1)$$

The L1 penalty can force some coefficients to zero and therefore performs feature selection.

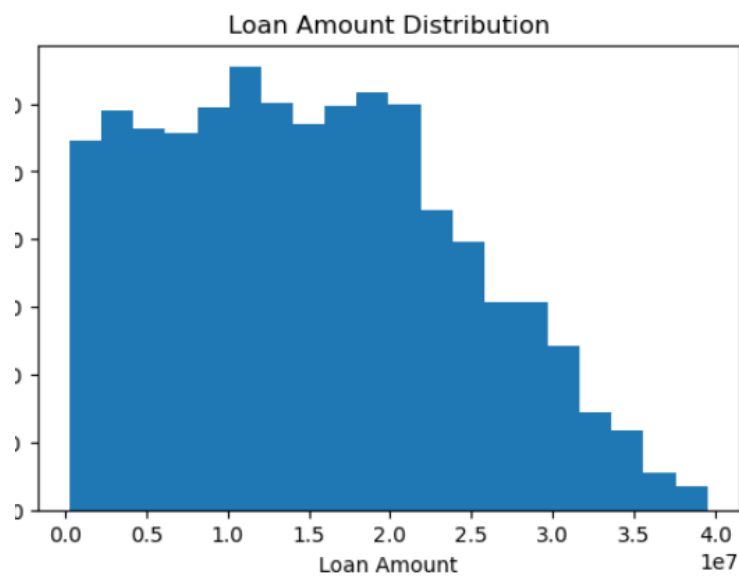
Elastic Net

Elastic Net combines L1 and L2 regularization:

$$\min_{\beta} (\|y - X\beta\|_2^2 + \alpha [l_1ratio\|\beta\|_1 + (1 - l_1ratio)\|\beta\|_2^2])$$

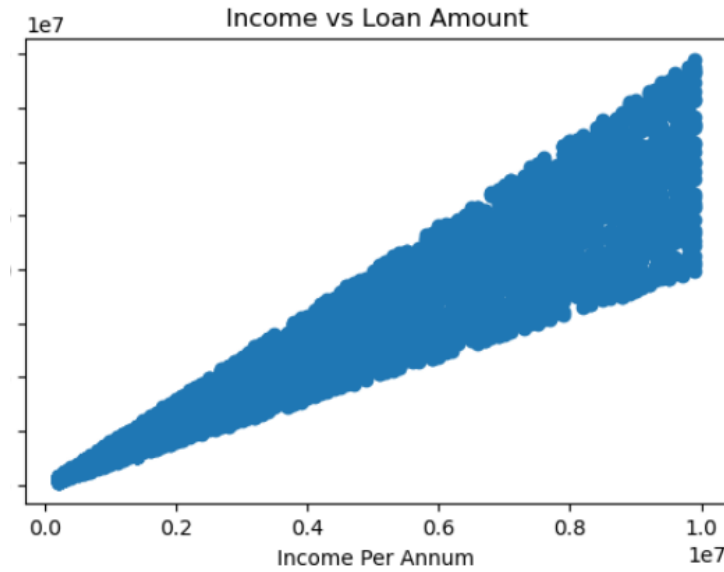
It provides a compromise between Ridge and Lasso behaviour.

Exploratory Data Analysis



```
figure(figsize=(6,4))
scatter(data["income_annum"], data["loan_amount"])
xlabel("Income Per Annum")
ylabel("Loan Amount")
title("Income vs Loan Amount")
```

Figure 1: Loan Amount Distribution



```
, X_test, y_train, y_test = train_test_split(
    t_size=0.2,
    dom_state=42
    Training Data :", X_train.shape)
```

Figure 2: Income per Annum vs Loan Amount

Baseline Model Performance

The four baseline regression models were trained and evaluated using MAE, MSE, RMSE and R^2 score.

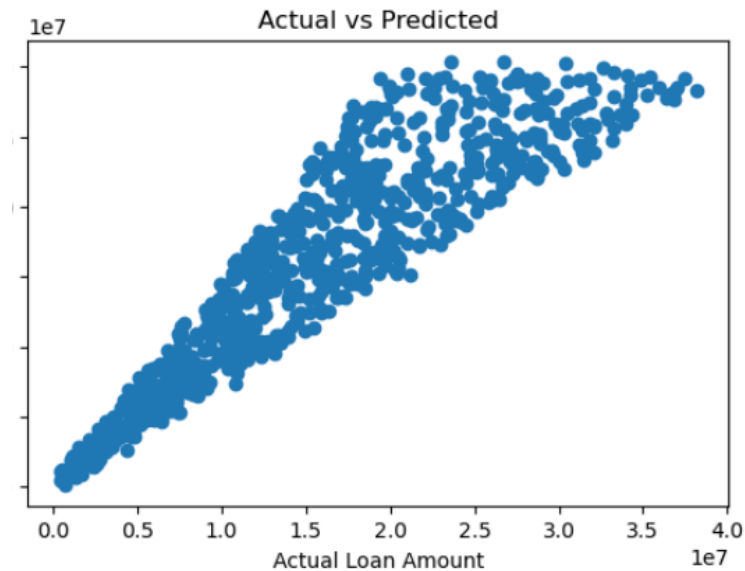
Table 1: Test Set Performance of Baseline Models

Model	MAE	MSE	RMSE	R^2
Linear Regression	2.598631×10^6	1.175502×10^{13}	3.428560×10^6	0.853401
Ridge Regression	2.598169×10^6	1.174876×10^{13}	3.427646×10^6	0.853479
Lasso Regression	2.598630×10^6	1.175502×10^{13}	3.428559×10^6	0.853401
Elastic Net	3.008482×10^6	1.439849×10^{13}	3.794535×10^6	0.820434

The Ridge model has the lowest MSE and RMSE and the highest R^2 among the four baseline models, although its improvement over Linear Regression is small.

Predicted vs Actual Values

The predicted versus actual plot shows how closely the model predictions follow the actual loan amounts.



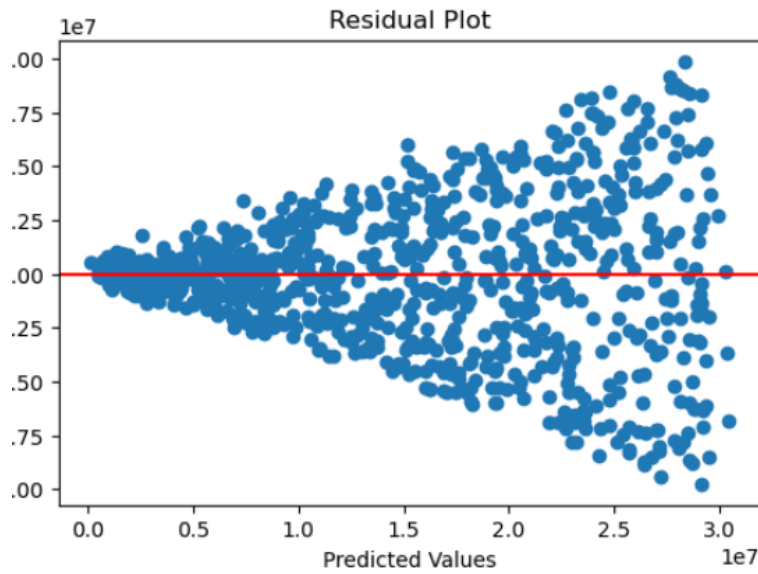
```
l = y_test - linear_pred
figure(figsize=(6,4))
scatter(linear_pred, residual)
line(0, color="red")
xlabel("Predicted Values")
```

Figure 3: Actual vs Predicted Loan Amount

Residual Analysis

Residuals were calculated as:

$$\text{Residual} = \text{Actual} - \text{Predicted}$$



```
ble = pd.DataFrame({
    'Model': [
        "Linear Regression",
        "Ridge Regression",
        "Lasso Regression",
        "Elastic Net"
    ],
    'Training Time': [
```

Figure 4: Residual Plot

The residual plot is used to examine whether prediction errors are randomly distributed around zero and to identify possible systematic patterns.

Training Time

The training times measured in the notebook were:

Table 2: Training Time	
Model	Training Time (s)
Linear Regression	0.024
Ridge Regression	0.008
Lasso Regression	0.019
Elastic Net	0.005

Hyperparameter Tuning

GridSearchCV with five-fold cross-validation was used for the regularized regression models.

Search Spaces

- Ridge: $\alpha \in \{0.01, 0.1, 1, 10, 100\}$
- Lasso: $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$
- Elastic Net: $\alpha \in \{0.01, 0.1, 1, 10\}$ and $l1_ratio \in \{0.2, 0.5, 0.8\}$

Table 3: Hyperparameter Tuning Results

Model	Best Parameters	Best CV R^2
Ridge Regression	$\alpha = 1$	0.863150
Lasso Regression	$\alpha = 10$	0.863148
Elastic Net	$\alpha = 0.01, l1_ratio = 0.8$	0.863126

Five-Fold Cross-Validation

The best hyperparameters obtained from GridSearchCV were used in five-fold cross-validation.

Table 4: Five-Fold Cross-Validation Performance

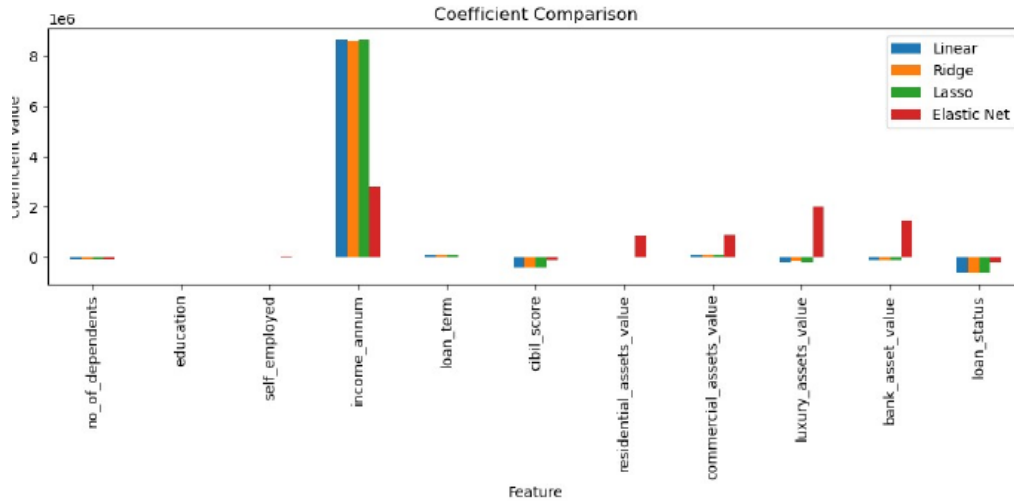
Model	Mean R^2
Linear Regression	0.861004
Ridge Regression	0.861004
Lasso Regression	0.861004
Elastic Net	0.860974

The cross-validation results are very close for Linear, Ridge and Lasso Regression, while Elastic Net gives a slightly lower mean R^2 .

Effect of Regularization on Coefficients

The coefficient table and coefficient comparison plot were generated to observe the effect of regularization on the model parameters.

```
t.title("Coefficient Comparison")
t.tight_layout()
t.show()
```



```
training_error = [
    mean_squared_error(y_train, linear.predict(X_train)),
    mean_squared_error(y_train, ridge.predict(X_train)),
    mean_squared_error(y_train, lasso.predict(X_train)),
    mean_squared_error(y_train, elastic.predict(X_train))
]
```

Figure 5: Coefficient Comparison

Regularization changes the magnitude of coefficients. Ridge shrinks coefficients through L2 regularization, Lasso can reduce coefficients towards zero, and Elastic Net combines both effects.

Overfitting and Underfitting Analysis

Training and validation MSE were compared for all four models.

Table 5: Training and Validation Error

Model	Training MSE	Validation MSE
Linear	1.114593×10^{13}	1.175502×10^{13}
Ridge	1.114601×10^{13}	1.174876×10^{13}
Lasso	1.114593×10^{13}	1.175502×10^{13}
Elastic Net	1.523658×10^{13}	1.439849×10^{13}

comparison

Model	Training Error	Validation Error
Linear	1.114593e+13	1.175502e+13
Ridge	1.114601e+13	1.174876e+13
Lasso	1.114593e+13	1.175502e+13
Elastic Net	1.523658e+13	1.439849e+13

```
comparison.set_index("Model").plot(
    kind="bar",
    figsize=(7,4)

    .ylabel("Mean Squared Error")
    .title("Training vs Validation Error")

    .tight_layout()

    .show())
```



Figure 6: Training Error vs Validation Error

The training and validation errors for Linear, Ridge and Lasso are close, indicating that these models do not show a large generalization gap. Elastic Net has higher errors in both training and validation, indicating a stronger regularization effect for the selected default model.

Bias–Variance Analysis

Linear Regression provides a simple baseline and generally has relatively low model complexity but can be affected by bias when the relationship is not adequately represented.

Ridge Regression reduces variance by shrinking coefficient magnitudes. In this experiment it provides a small improvement in test MSE and R^2 over the baseline.

Lasso Regression adds sparsity through L1 regularization. It can remove less useful features by driving coefficients toward zero.

Elastic Net combines L1 and L2 regularization. It can provide both coefficient shrinkage and sparsity, but the default Elastic Net model in this experiment had a lower test R^2 than the other models.

Observations

- Linear, Ridge, Lasso and Elastic Net regression models were successfully implemented.
- Ridge Regression achieved the best baseline test R^2 of 0.853479.
- Ridge also had the lowest baseline MSE and RMSE.
- GridSearchCV selected $\alpha = 1$ for Ridge, $\alpha = 10$ for Lasso, and $\alpha = 0.01$, $l1_ratio = 0.8$ for Elastic Net.
- The best cross-validation R^2 was obtained by Ridge at 0.863150.
- Regularization produced only a small improvement over the Linear Regression baseline for this dataset.
- Lasso provides feature sparsity, while Ridge primarily shrinks coefficient magnitudes.
- Elastic Net combines the properties of Lasso and Ridge.
- The training and validation errors are useful for identifying generalization behaviour.

Conclusion

Linear Regression, Ridge Regression, Lasso Regression and Elastic Net Regression were implemented for predicting loan amounts. The data was preprocessed by encoding categorical variables and standardizing the input features. GridSearchCV with five-fold cross-validation was used to tune the regularization parameters.

Among the baseline models, Ridge Regression produced the best test performance with an R^2 score of 0.853479 and the lowest MSE and RMSE. GridSearchCV selected $\alpha = 1$ for Ridge, $\alpha = 10$ for Lasso and $\alpha = 0.01$ with $l1_ratio = 0.8$ for Elastic Net. The cross-validation results were close for Linear, Ridge and Lasso, showing that regularization produced a relatively small performance change for this dataset. The experiment demonstrates the role of regularization in controlling model complexity, reducing variance and, in the case of Lasso, encouraging feature sparsity.

Complete Python Code

The following section contains **all 31 code cells from the uploaded Assignment 3 notebook**, preserved in their original order.

```
% Code Cell 1
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score

from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

% Code Cell 2
data = pd.read_csv("loan_data.csv")

data.columns = data.columns.str.strip()

data.head()

% Code Cell 3
print("Shape :", data.shape)

print("\nColumn Names")
print(data.columns)

print("\nData Types")
print(data.dtypes)

data.info()

% Code Cell 4
```

```

data.describe()

% Code Cell 5
data.isnull().sum()

% Code Cell 6
data = data.drop("loan_id", axis=1)

encoder = LabelEncoder()

data["education"] = encoder.fit_transform(data["education"])
data["self_employed"] = encoder.fit_transform(data["self_employed"])
data["loan_status"] = encoder.fit_transform(data["loan_status"])

data.head()

% Code Cell 7
X = data.drop("loan_amount", axis=1)

y = data["loan_amount"]

% Code Cell 8
scaler = StandardScaler()

X = scaler.fit_transform(X)

% Code Cell 9
plt.figure(figsize=(6,4))

plt.hist(y, bins=20)

plt.title("Loan Amount Distribution")

plt.xlabel("Loan Amount")

plt.ylabel("Frequency")

plt.show()

% Code Cell 10
plt.figure(figsize=(6,4))

plt.scatter(data["income_annum"], data["loan_amount"])

plt.xlabel("Income Per Annum")

plt.ylabel("Loan Amount")

```

```

plt.title("Income vs Loan Amount")

plt.show()

% Code Cell 11
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data :", X_train.shape)
print("Testing Data :", X_test.shape)

% Code Cell 12
linear = LinearRegression()

start = time.time()

linear.fit(X_train, y_train)

linear_time = time.time() - start

linear_pred = linear.predict(X_test)

% Code Cell 13
ridge = Ridge()

start = time.time()

ridge.fit(X_train, y_train)

ridge_time = time.time() - start

ridge_pred = ridge.predict(X_test)

% Code Cell 14
lasso = Lasso()

start = time.time()

lasso.fit(X_train, y_train)

lasso_time = time.time() - start

```

```

lasso_pred = lasso.predict(X_test)

% Code Cell 15
elastic = ElasticNet()

start = time.time()

elastic.fit(X_train, y_train)

elastic_time = time.time() - start

elastic_pred = elastic.predict(X_test)

% Code Cell 16
def calculate_metrics(actual, predicted):

    mae = mean_absolute_error(actual, predicted)
    mse = mean_squared_error(actual, predicted)
    rmse = np.sqrt(mse)
    r2 = r2_score(actual, predicted)

    return mae, mse, rmse, r2

% Code Cell 17
linear_result = calculate_metrics(y_test, linear_pred)
ridge_result = calculate_metrics(y_test, ridge_pred)
lasso_result = calculate_metrics(y_test, lasso_pred)
elastic_result = calculate_metrics(y_test, elastic_pred)

performance = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Ridge Regression",
        "Lasso Regression",
        "Elastic Net"
    ],
    "MAE": [
        linear_result[0],
        ridge_result[0],
        lasso_result[0],
        elastic_result[0]
    ],
    "MSE": [
        linear_result[1],
        ridge_result[1],
        lasso_result[1],
        elastic_result[1]
    ]
})

```

```

    ],
    "RMSE": [
        linear_result[2],
        ridge_result[2],
        lasso_result[2],
        elastic_result[2]
    ],
    "R2 Score": [
        linear_result[3],
        ridge_result[3],
        lasso_result[3],
        elastic_result[3]
    ]
})

performance

% Code Cell 18
plt.figure(figsize=(6,4))

plt.scatter(y_test, linear_pred)

plt.xlabel("Actual Loan Amount")
plt.ylabel("Predicted Loan Amount")

plt.title("Actual vs Predicted")

plt.show()

% Code Cell 19
residual = y_test - linear_pred

plt.figure(figsize=(6,4))

plt.scatter(linear_pred, residual)

plt.axhline(0, color="red")

plt.xlabel("Predicted Values")
plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.show()

% Code Cell 20
time_table = pd.DataFrame({

```

```

        "Model": [
            "Linear Regression",
            "Ridge Regression",
            "Lasso Regression",
            "Elastic Net"
        ],
        "Training Time": [
            linear_time,
            ridge_time,
            lasso_time,
            elastic_time
        ]
    })

time_table

% Code Cell 21
ridge_parameters = {
    "alpha": [0.01, 0.1, 1, 10, 100]
}

ridge_search = GridSearchCV(
    Ridge(),
    ridge_parameters,
    cv=5,
    scoring="r2"
)

ridge_search.fit(X_train, y_train)

print(ridge_search.best_params_)
print(ridge_search.best_score_)

% Code Cell 22
lasso_parameters = {
    "alpha": [0.001, 0.01, 0.1, 1, 10]
}

lasso_search = GridSearchCV(
    Lasso(max_iter=10000),
    lasso_parameters,
    cv=5,
    scoring="r2"
)

lasso_search.fit(X_train, y_train)

```

```

print(lasso_search.best_params_)
print(lasso_search.best_score_)

% Code Cell 23
elastic_parameters = {
    "alpha": [0.01, 0.1, 1, 10],
    "l1_ratio": [0.2, 0.5, 0.8]
}

elastic_search = GridSearchCV(
    ElasticNet(max_iter=10000),
    elastic_parameters,
    cv=5,
    scoring="r2"
)

elastic_search.fit(X_train, y_train)

print(elastic_search.best_params_)
print(elastic_search.best_score_)

% Code Cell 24
tuning_table = pd.DataFrame({
    "Model": [
        "Ridge Regression",
        "Lasso Regression",
        "Elastic Net"
    ],
    "Search Method": [
        "GridSearchCV",
        "GridSearchCV",
        "GridSearchCV"
    ],
    "Best Parameters": [
        str(ridge_search.best_params_),
        str(lasso_search.best_params_),
        str(elastic_search.best_params_)
    ],
    "Best CV R2": [
        ridge_search.best_score_,
        lasso_search.best_score_,
        elastic_search.best_score_
    ]
})

tuning_table

```

```

% Code Cell 25
linear_cv = cross_val_score(
    LinearRegression(),
    X,
    Y,
    cv=5,
    scoring="r2"
)

ridge_cv = cross_val_score(
    Ridge(alpha=ridge_search.best_params_["alpha"]),
    X,
    Y,
    cv=5,
    scoring="r2"
)

lasso_cv = cross_val_score(
    Lasso(alpha=lasso_search.best_params_["alpha"], max_iter=10000),
    X,
    Y,
    cv=5,
    scoring="r2"
)

elastic_cv = cross_val_score(
    ElasticNet(
        alpha=elastic_search.best_params_["alpha"],
        l1_ratio=elastic_search.best_params_["l1_ratio"],
        max_iter=10000
    ),
    X,
    Y,
    cv=5,
    scoring="r2"
)

% Code Cell 26
cv_table = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Ridge Regression",
        "Lasso Regression",
        "Elastic Net"
    ],
    "R2": [
        linear_cv.mean(),

```

```

        ridge_cv.mean(),
        lasso_cv.mean(),
        elastic_cv.mean()
    ]
}))

cv_table

% Code Cell 27
coefficient_table = pd.DataFrame({
    "Feature": data.drop("loan_amount", axis=1).columns,
    "Linear": linear.coef_,
    "Ridge": ridge.coef_,
    "Lasso": lasso.coef_,
    "Elastic Net": elastic.coef_
})

coefficient_table

% Code Cell 28
coefficient_table.set_index("Feature").plot(
    kind="bar",
    figsize=(10,5)
)

plt.ylabel("Coefficient Value")
plt.title("Coefficient Comparison")

plt.tight_layout()

plt.show()

% Code Cell 29
training_error = [
    mean_squared_error(y_train, linear.predict(X_train)),
    mean_squared_error(y_train, ridge.predict(X_train)),
    mean_squared_error(y_train, lasso.predict(X_train)),
    mean_squared_error(y_train, elastic.predict(X_train))
]

validation_error = [
    linear_result[1],
    ridge_result[1],
    lasso_result[1],
    elastic_result[1]
]

```

```

comparison = pd.DataFrame({
    "Model": [
        "Linear",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],
    "Training Error": training_error,
    "Validation Error": validation_error
})

comparison

% Code Cell 30
comparison.set_index("Model").plot(
    kind="bar",
    figsize=(7,4)
)

plt.ylabel("Mean Squared Error")
plt.title("Training vs Validation Error")

plt.tight_layout()

plt.show()

% Code Cell 31

```

References

- Scikit-learn Documentation – Linear Models.
- Scikit-learn Documentation – Hyperparameter Optimization.
- Loan Amount Dataset.